a personal memory layer for repetitive web forms

MemoryBank Finalized Architecture Document

1. Product Identity

Name: MemoryBank
Type: Local-first Chrome extension
Purpose: Remember previously entered answers on web forms and help the user reuse them later.

MemoryBank is intentionally **not** an AI product. It does not generate answers, rewrite answers, summarize answers, or depend on any token-based model. It is designed to be:

- lightweight
- privacy-first
- simple
- predictable
- local-only

Its value is memory retention, not intelligence.

---

2. Product Vision

MemoryBank should feel like a silent memory layer for forms.

When a user encounters a question they have answered before, MemoryBank should:

1. detect the field
2. infer the question being asked
3. check whether a prior memory exists
4. suggest or apply that memory depending on settings
5. offer to save new answers when useful

The product should remain compact and focused. It should not evolve into a broad job-search platform or a bloated browser assistant.

---

3. Product Principles

Core principles

- **No AI**
- **No cloud requirement**
- **No accounts**
- **No tracking dashboards**
- **No unnecessary automation**
- **User control first**
- **Everything stored locally**
- **Fast and minimal UI**

Design philosophy

MemoryBank should prefer being:

- understandable over clever
- reliable over magical
- minimal over feature-heavy
- transparent over hidden

---

4. Scope

In scope for v1

- detect common form fields
- infer question text from the page
- match against saved memories
- suggest or apply saved answers
- prompt to save new answers
- enable or disable globally
- configure apply behavior
- browse, edit, and delete saved memories
- support per-site behavior overrides

---

5. Supported Field Types

MemoryBank v1 should support the following field types:

- text input
- textarea
- select dropdown
- checkbox
- radio group

Future possibility

- richer support for custom widgets on difficult sites

Important realism note

MemoryBank should aim to support **most standard and many custom fields**, but it should not promise perfect compatibility with every possible site implementation.

---

6. Primary User Flows

Flow A: Detect and suggest

1. User opens a page with form fields.
2. MemoryBank scans the page.
3. A field is identified.
4. The likely question is extracted.
5. A match is searched in local memory.
6. If a match exists, MemoryBank shows a suggestion or prompt.

Flow B: Apply prior answer

1. User sees a suggestion.
2. User accepts the suggestion, or the system auto-applies based on settings.
3. MemoryBank fills the field.
4. The page receives proper DOM events.
5. Usage stats are updated.

Flow C: Save a new answer

1. User manually fills a field.
2. MemoryBank detects meaningful completion.
3. A save prompt appears.
4. User approves saving.
5. A new memory is stored locally.

Flow D: Review and edit memories

1. User opens the dashboard.
2. User browses or searches memories.
3. User edits question text and answer content.
4. The updated memory is saved locally.

---

7. Runtime Architecture

MemoryBank is split into multiple runtimes.

A. Content runtime

Runs inside the active webpage.

Responsibilities:

- scan the DOM for form fields
- extract field metadata
- show inline prompts and indicators
- apply values into fields
- detect manual user entry
- observe dynamic page changes

B. Background runtime

Runs as the extension coordinator.

Responsibilities:

- central message routing
- memory lookup orchestration
- settings resolution
- repository access
- site rule handling
- usage updates

C. Popup runtime

Quick control surface.

Responsibilities:

- global enable or disable
- current mode selection
- current-site controls
- status display

D. Options/dashboard runtime

Full management UI.

Responsibilities:

- browse memories
- edit and delete memories
- configure settings
- manage site rules

---

8. Architectural Style

MemoryBank should use a **layered architecture with MVVM-inspired presentation**.

Layers

1. **Presentation layer**
2. **Application layer**
3. **Domain layer**
4. **Infrastructure layer**

This keeps the project explainable, testable, and modular.

---

9. Layer Responsibilities

9.1 Presentation layer

Owns all UI that MemoryBank renders.

Includes:

- popup views
- options/dashboard views
- inline content prompts

- indicators and menus

Responsibility: present state and handle user interactions.

9.2 Application layer

Owns orchestration and use cases.

Responsibility:

- normalize question inputs
- match memories
- decide whether to suggest, prompt, or auto-apply
- save and update memories
- resolve site-specific behavior

9.3 Domain layer

Owns business meaning.

Responsibility:

- define what a memory is
- define what a detected question is
- define what a match result is
- define confidence levels and decision types

9.4 Infrastructure layer

Owns browser and storage implementation details.

Responsibility:

- Chrome APIs
- local storage wrappers
- DOM scanning
- event dispatching
- mutation observation
- messaging

---

10. Folder Structure

```
memorybank/
  src/
```

```
  domain/
    models/
    enums/
    valueObjects/
    rules/
  application/
    services/
    coordinators/
    useCases/
    interfaces/
  infrastructure/
    browser/
    storage/
    dom/
    messaging/
    logging/
  presentation/
    popup/
    options/
    content/
    shared/
  background/
    handlers/
    background.ts
  shared/
    constants/
    types/
    utils/
manifest.json
```

---

11. Final Naming Baseline

Models

- `MemoryEntry`
- `AnswerPayload`
- `QuestionSignature`
- `DetectedFieldDescriptor`
- `FieldContext`
- `MatchResult`

- `UserSettings`
- `SiteRule`

Enums

- `FieldType`
- `AnswerType`
- `ApplyMode`
- `ConfidenceLevel`
- `SiteRuleMode`

Services

- `QuestionNormalizationService`
- `MemoryMatchingService`
- `MemoryDecisionService`
- `MemorySaveService`
- `SettingsResolutionService`
- `MemorySearchService`
- `DomScannerService`
- `DomQuestionLocatorService`
- `DomFieldWriterService`
- `DomEventDispatcherService`
- `MutationObserverService`
- `ChromeStorageService`
- `ChromeMessagingService`
- `LoggerService`

Repositories

- `MemoryRepository`
- `SettingsRepository`
- `SiteRulesRepository`

Coordinators

- `PageAnalysisCoordinator`
- `FieldInteractionCoordinator`
- `MemoryLifecycleCoordinator`

Views

- `PopupHomeView`
- `MemoryDashboardView`
- `MemoryListView`
- `MemoryEditView`
- `SettingsView`
- `SiteRulesView`
- `ApplyPromptView`
- `SaveMemoryPromptView`
- `FieldSuggestionBadgeView`

ViewModels

- `PopupHomeViewModel`
- `MemoryDashboardViewModel`
- `MemoryListViewModel`
- `MemoryEditViewModel`
- `SettingsViewModel`
- `SiteRulesViewModel`
- `ApplyPromptViewModel`
- `SaveMemoryPromptViewModel`
- `FieldSuggestionBadgeViewModel`

---

12. Domain Model Spec

12.1 MemoryEntry

Represents one saved memory.

Fields:

- `id: string`
- `questionText: string`
- `normalizedQuestionText: string`
- `answer: AnswerPayload`
- `answerType: AnswerType`
- `tags: string[]`

- `createdAt: string`
- `updatedAt: string`
- `usageCount: number`
- `lastUsedAt?: string`
- `enabled: boolean`
- `sourceHosts: string[]`

12.2 AnswerPayload

Represents the stored answer value.

Fields:

- `textValue?: string`
- `booleanValue?: boolean`
- `selectValue?: string`
- `multiSelectValues?: string[]`

12.3 QuestionSignature

Represents the interpreted identity of a detected question.

Fields:

- `rawQuestionText: string`
- `normalizedQuestionText: string`
- `keywords: string[]`
- `fieldType: FieldType`
- `placeholderText?: string`
- `sectionText?: string`
- `optionTexts?: string[]`

12.4 DetectedFieldDescriptor

Represents a field on the page in a DOM-agnostic format.

Fields:

- `fieldId: string`
- `fieldType: FieldType`
- `questionText: string`
- `placeholderText?: string`

- `isRequired: boolean`
- `optionTexts?: string[]`
- `hostName: string`
- `sectionText?: string`

12.5 FieldContext

Represents surrounding page context.

Fields:

- `hostName: string`
- `pageTitle?: string`
- `sectionText?: string`
- `nearbyText?: string`

12.6 MatchResult

Represents the result of matching a question to memory.

Fields:

- `memoryId?: string`
- `confidenceScore: number`
- `confidenceLevel: ConfidenceLevel`
- `reason: string`
- `matchedQuestionText?: string`
- `answer?: AnswerPayload`
- `hasMatch: boolean`
- `alternateMemoryIds?: string[]`

12.7 UserSettings

Represents global extension behavior.

Fields:

- `isEnabled: boolean`
- `defaultApplyMode: ApplyMode`
- `promptToSaveNewAnswers: boolean`
- `autoApplyConfidenceThreshold: number`
- `showInlineIndicators: boolean`

- `siteRulesEnabled: boolean`

12.8 SiteRule

Represents host-specific behavior overrides.

Fields:

- `hostName: string`
- `isEnabled: boolean`
- `overrideApplyMode?: ApplyMode`

---

13. Enums

FieldType

- `Text`
- `TextArea`
- `Select`
- `Checkbox`
- `Radio`
- `Unknown`

AnswerType

- `Text`
- `Boolean`
- `SelectChoice`
- `MultiSelect`
- `Unknown`

ApplyMode

- `Disabled`
- `SuggestOnly`
- `AskBeforeApply`
- `AutoApply`

ConfidenceLevel

- `None`
- `Low`
- `Medium`
- `High`

SiteRuleMode

- `Default`
- `Disabled`
- `Override`

---

14. Service Responsibilities

QuestionNormalizationService

Normalizes raw question text into a stable comparable form.

Responsibilities:

- lowercase text
- trim whitespace
- remove or normalize punctuation
- collapse repeated spaces
- optionally extract useful keywords

MemoryMatchingService

Matches a `QuestionSignature` against all saved memories.

Responsibilities:

- exact normalized matching
- near-exact string similarity
- keyword overlap fallback
- field type compatibility checks

MemoryDecisionService

Determines what MemoryBank should do with a match.

Possible outcomes:

- do nothing
- suggest
- prompt
- auto-apply

MemorySaveService

Creates and stores new memories.

Responsibilities:

- normalize question text
- infer answer type
- detect duplicates
- persist memory metadata

SettingsResolutionService

Resolves effective behavior for the current site.

Responsibilities:

- combine global settings and site rule overrides
- determine effective apply mode
- determine whether MemoryBank is active on the page

MemorySearchService

Supports dashboard filtering and search.

DomScannerService

Scans the page for relevant form controls.

DomQuestionLocatorService

Extracts the likely question for a field using:

- label text
- aria attributes
- placeholder
- nearby sibling text
- section headings
- fieldset and legend text

DomFieldWriterService

Writes an answer into a field.

DomEventDispatcherService

Dispatches correct DOM events after writing values.

MutationObserverService

Watches for dynamic form changes and rerenders.

ChromeStorageService

Wraps local storage operations.

ChromeMessagingService

Wraps cross-runtime messaging.

LoggerService

Captures debug logging in development.

---

15. Repository Responsibilities

MemoryRepository

- get all memories
- get memory by id
- save memory
- update memory
- delete memory
- search memories

SettingsRepository

- load settings
- save settings

SiteRulesRepository

- load rules
- get rule by host
- save rule
- delete rule

---

16. View Responsibilities

Popup views

- `PopupHomeView`
- `PopupStatusCard`
- `PopupModeSelector`
- `PopupSiteToggleCard`

Responsibilities:

- enable or disable MemoryBank
- switch apply mode
- toggle current site behavior
- display active state

Dashboard views

- `MemoryDashboardView`
- `MemoryListView`
- `MemoryEditView`
- `SettingsView`
- `SiteRulesView`

Responsibilities:

- browse and search memories
- edit or delete memories
- manage global settings
- manage site-specific rules

In-page views

- `FieldSuggestionBadgeView`
- `ApplyPromptView`
- `SaveMemoryPromptView`

- `FieldStateIndicatorView`

Responsibilities:

- show a previous memory is available
- ask whether to apply a memory
- ask whether to save a new answer
- show field status subtly

---

17. Main Data Flows

Data Flow 1: Analyze and suggest

1. Content runtime scans the page.
2. A field is found.
3. Question text is extracted.
4. A `DetectedFieldDescriptor` is built.
5. Background runtime resolves effective settings.
6. Question normalization runs.
7. Memory matching runs.
8. Decision service determines the action.
9. Content runtime shows suggestion, prompt, or auto-apply state.

Data Flow 2: Apply a memory

1. User accepts a suggestion, or auto-apply is triggered.
2. Content runtime writes the value.
3. DOM events are dispatched.
4. Usage metadata is updated.

Data Flow 3: Save a new memory

1. User manually fills a field.
2. Content runtime detects meaningful completion.
3. Save prompt is shown.
4. User confirms.
5. Save service creates a new `MemoryEntry`.
6. Repository persists it locally.

Data Flow 4: Edit a memory

1. Dashboard loads memory list.

2. User selects a memory.
3. User edits question and answer.
4. Update flow normalizes the updated question.
5. Repository persists the new state.

---

18. Message Boundaries

MemoryBank should communicate through clear message contracts between runtimes.

Example message categories

- analyze page
- request match for field
- apply memory
- save memory
- load settings
- update settings
- fetch memories
- update memory
- delete memory
- fetch site rule
- update site rule

The project should avoid ad-hoc message payloads. Message shapes should be strongly typed from the beginning.

---

19. Matching Strategy

Final v1 decision

MemoryBank will use a **non-AI, local, explainable matching strategy**.

Matching stages

1. exact normalized match
2. near-exact string similarity
3. keyword overlap fallback
4. field type compatibility filter

Why this is the right choice

- keeps the product lightweight
- preserves privacy
- avoids model cost
- is easier to debug
- is easier to explain
- avoids AI fatigue entirely

---

20. Settings Spec

Global settings

- enabled
- default apply mode
- prompt to save new answers
- auto-apply confidence threshold
- show inline indicators
- enable site rules

Per-site settings

- site enabled or disabled
- site-specific apply mode override

Supported apply modes

- `SuggestOnly`
- `AskBeforeApply`
- `AutoApply`

Recommended default

- `AskBeforeApply`

This is the best balance between usefulness and safety.

---

21. Pain Points and Mitigations

Pain point 1: Question extraction is messy

Many sites do not properly associate labels with fields.

Mitigation:

- layered extraction strategy
- fallback heuristics
- keep extraction logic isolated in `DomQuestionLocatorService`
- add site-specific improvements later if needed

Pain point 2: Wrong autofills

A weak match can produce a bad suggestion or fill.

Mitigation:

- default to `AskBeforeApply`
- use confidence levels
- show source question before applying
- gate auto-apply by threshold

Pain point 3: Duplicate memories

Users may save similar memories many times.

Mitigation:

- duplicate checks during save
- editing support in dashboard
- merge support later if needed

Pain point 4: Noisy save prompts

Prompting too often can become annoying.

Mitigation:

- only prompt on meaningful completion
- minimum content thresholds
- prompt on blur or stable completion points
- ignore obviously trivial fields where appropriate

Pain point 5: Dynamic form rerenders

Modern sites often rerender fields.

Mitigation:

- mutation observer
- debounced re-analysis
- field identity strategy resilient to DOM changes

Pain point 6: Huge content script risk

It is easy for this kind of project to collapse into one giant content script.

Mitigation:

- keep content layer thin
- keep decisions in application layer
- keep storage behind repositories

---

22. Implementation Guardrails

These rules should remain true throughout development.

Rule 1

The content layer may detect and apply, but it should not own business logic.

Rule 2

The domain layer must not depend on Chrome APIs or raw DOM elements.

Rule 3

Storage access should happen through repositories only.

Rule 4

Settings resolution should happen in one place, not in multiple runtimes.

Rule 5

Matching behavior must remain explainable.

Rule 6

Any new feature must be evaluated against the lightweight and local-first philosophy.

---

23. MVP Build Order

Phase 1: Foundation

- manifest setup
- runtime messaging skeleton
- storage wrappers
- repositories
- popup enable or disable control
- popup mode selection

Phase 2: Detection

- DOM scanner
- question locator
- descriptor creation
- page analysis coordinator

Phase 3: Memory core

- normalization service
- matching service
- match result model
- settings resolution

Phase 4: Apply flow

- inline suggestion UI
- apply prompt
- field writing service
- DOM event dispatch support

Phase 5: Save flow

- manual completion detection
- save prompt
- save service
- duplicate handling basics

Phase 6: Dashboard

- memory listing
- search and filtering
- edit and delete memory
- settings page
- site rules page

Phase 7: Hardening

- dynamic form support improvements
- extraction heuristics refinement
- compatibility improvements on difficult sites

---

24. Final Product Positioning

MemoryBank should be positioned as:

> A lightweight, local-first form memory extension that remembers how you answered questions before and helps you reuse those answers quickly.

It should not be positioned as:

- an AI copilot
- a career platform
- a job search suite
- an autonomous browser agent

That focus is part of the product's strength.

---

25. Final Recommendation

The project should move into coding only after preserving these decisions:

- no AI anywhere
- local-only storage
- simple matching strategy
- thin content layer
- layered architecture
- editability and transparency over cleverness
- `AskBeforeApply` as default
- compact UI and limited scope

If MemoryBank stays inside those boundaries, it has a strong chance of being both useful and realistically shippable.

MemoryBank Finalized Architecture Document

1. Product Identity

Name: MemoryBank
Type: Local-first Chrome extension
Purpose: Remember previously entered answers on web forms and help the user reuse them later.

MemoryBank is intentionally **not** an AI product. It does not generate answers, rewrite answers, summarize answers, or depend on any token-based model. It is designed to be:

- lightweight
- privacy-first
- simple
- predictable
- local-only

Its value is memory retention, not intelligence.

---

2. Product Vision

MemoryBank should feel like a silent memory layer for forms.

When a user encounters a question they have answered before, MemoryBank should:

1. detect the field
2. infer the question being asked
3. check whether a prior memory exists
4. suggest or apply that memory depending on settings
5. offer to save new answers when useful

The product should remain compact and focused. It should not evolve into a broad job-search platform or a bloated browser assistant.

---

3. Product Principles

Core principles

- **No AI**
- **No cloud requirement**
- **No accounts**
- **No tracking dashboards**
- **No unnecessary automation**

- **User control first**
- **Everything stored locally**
- **Fast and minimal UI**

Design philosophy

MemoryBank should prefer being:

- understandable over clever
- reliable over magical
- minimal over feature-heavy
- transparent over hidden

---

4. Scope

In scope for v1

- detect common form fields
- infer question text from the page
- match against saved memories
- suggest or apply saved answers
- prompt to save new answers
- enable or disable globally
- configure apply behavior
- browse, edit, and delete saved memories
- support per-site behavior overrides

---

5. Supported Field Types

MemoryBank v1 should support the following field types:

- text input
- textarea
- select dropdown
- checkbox
- radio group

Future possibility

- richer support for custom widgets on difficult sites

Important realism note

MemoryBank should aim to support **most standard and many custom fields**, but it should not promise perfect compatibility with every possible site implementation.

---

6. Primary User Flows

Flow A: Detect and suggest

1. User opens a page with form fields.
2. MemoryBank scans the page.
3. A field is identified.
4. The likely question is extracted.
5. A match is searched in local memory.
6. If a match exists, MemoryBank shows a suggestion or prompt.

Flow B: Apply prior answer

1. User sees a suggestion.
2. User accepts the suggestion, or the system auto-applies based on settings.
3. MemoryBank fills the field.
4. The page receives proper DOM events.
5. Usage stats are updated.

Flow C: Save a new answer

1. User manually fills a field.
2. MemoryBank detects meaningful completion.
3. A save prompt appears.
4. User approves saving.
5. A new memory is stored locally.

Flow D: Review and edit memories

1. User opens the dashboard.
2. User browses or searches memories.
3. User edits question text and answer content.
4. The updated memory is saved locally.

---

7. Runtime Architecture

MemoryBank is split into multiple runtimes.

A. Content runtime

Runs inside the active webpage.

Responsibilities:

- scan the DOM for form fields
- extract field metadata
- show inline prompts and indicators
- apply values into fields
- detect manual user entry
- observe dynamic page changes

B. Background runtime

Runs as the extension coordinator.

Responsibilities:

- central message routing
- memory lookup orchestration
- settings resolution
- repository access
- site rule handling
- usage updates

C. Popup runtime

Quick control surface.

Responsibilities:

- global enable or disable
- current mode selection
- current-site controls
- status display

D. Options/dashboard runtime

Full management UI.

Responsibilities:

- browse memories
- edit and delete memories
- configure settings
- manage site rules

---

8. Architectural Style

MemoryBank should use a **layered architecture with MVVM-inspired presentation**.

Layers

1. **Presentation layer**
2. **Application layer**
3. **Domain layer**
4. **Infrastructure layer**

This keeps the project explainable, testable, and modular.

---

9. Layer Responsibilities

9.1 Presentation layer

Owns all UI that MemoryBank renders.

Includes:

- popup views
- options/dashboard views
- inline content prompts
- indicators and menus

Responsibility: present state and handle user interactions.

9.2 Application layer

Owns orchestration and use cases.

Responsibility:

- normalize question inputs
- match memories
- decide whether to suggest, prompt, or auto-apply
- save and update memories
- resolve site-specific behavior

9.3 Domain layer

Owns business meaning.

Responsibility:

- define what a memory is
- define what a detected question is
- define what a match result is
- define confidence levels and decision types

9.4 Infrastructure layer

Owns browser and storage implementation details.

Responsibility:

- Chrome APIs
- local storage wrappers
- DOM scanning
- event dispatching
- mutation observation
- messaging

---

10. Folder Structure

```
memorybank/

  src/

    domain/

      models/

      enums/

      valueObjects/
```

```
  rules/
application/
  services/
  coordinators/
  useCases/
  interfaces/
infrastructure/
  browser/
  storage/
  dom/
  messaging/
  logging/
presentation/
  popup/
  options/
  content/
  shared/
background/
  handlers/
  background.ts
shared/
  constants/
  types/
  utils/
```

manifest.json

---

11. Final Naming Baseline

Models

- `MemoryEntry`
- `AnswerPayload`
- `QuestionSignature`
- `DetectedFieldDescriptor`
- `FieldContext`
- `MatchResult`
- `UserSettings`
- `SiteRule`

Enums

- `FieldType`
- `AnswerType`
- `ApplyMode`
- `ConfidenceLevel`
- `SiteRuleMode`

Services

- `QuestionNormalizationService`
- `MemoryMatchingService`
- `MemoryDecisionService`
- `MemorySaveService`
- `SettingsResolutionService`
- `MemorySearchService`
- `DomScannerService`
- `DomQuestionLocatorService`
- `DomFieldWriterService`
- `DomEventDispatcherService`
- `MutationObserverService`

- `ChromeStorageService`
- `ChromeMessagingService`
- `LoggerService`

Repositories

- `MemoryRepository`
- `SettingsRepository`
- `SiteRulesRepository`

Coordinators

- `PageAnalysisCoordinator`
- `FieldInteractionCoordinator`
- `MemoryLifecycleCoordinator`

Views

- `PopupHomeView`
- `MemoryDashboardView`
- `MemoryListView`
- `MemoryEditView`
- `SettingsView`
- `SiteRulesView`
- `ApplyPromptView`
- `SaveMemoryPromptView`
- `FieldSuggestionBadgeView`

ViewModels

- `PopupHomeViewModel`
- `MemoryDashboardViewModel`
- `MemoryListViewModel`
- `MemoryEditViewModel`
- `SettingsViewModel`
- `SiteRulesViewModel`
- `ApplyPromptViewModel`
- `SaveMemoryPromptViewModel`
- `FieldSuggestionBadgeViewModel`

---

12. Domain Model Spec

12.1 MemoryEntry

Represents one saved memory.

Fields:

- `id: string`
- `questionText: string`
- `normalizedQuestionText: string`
- `answer: AnswerPayload`
- `answerType: AnswerType`
- `tags: string[]`
- `createdAt: string`
- `updatedAt: string`
- `usageCount: number`
- `lastUsedAt?: string`
- `enabled: boolean`
- `sourceHosts: string[]`

12.2 AnswerPayload

Represents the stored answer value.

Fields:

- `textValue?: string`
- `booleanValue?: boolean`
- `selectValue?: string`
- `multiSelectValues?: string[]`

12.3 QuestionSignature

Represents the interpreted identity of a detected question.

Fields:

- `rawQuestionText: string`
- `normalizedQuestionText: string`
- `keywords: string[]`
- `fieldType: FieldType`

- `placeholderText?: string`
- `sectionText?: string`
- `optionTexts?: string[]`

12.4 DetectedFieldDescriptor

Represents a field on the page in a DOM-agnostic format.

Fields:

- `fieldId: string`
- `fieldType: FieldType`
- `questionText: string`
- `placeholderText?: string`
- `isRequired: boolean`
- `optionTexts?: string[]`
- `hostName: string`
- `sectionText?: string`

12.5 FieldContext

Represents surrounding page context.

Fields:

- `hostName: string`
- `pageTitle?: string`
- `sectionText?: string`
- `nearbyText?: string`

12.6 MatchResult

Represents the result of matching a question to memory.

Fields:

- `memoryId?: string`
- `confidenceScore: number`
- `confidenceLevel: ConfidenceLevel`
- `reason: string`
- `matchedQuestionText?: string`
- `answer?: AnswerPayload`

- `hasMatch: boolean`
- `alternateMemoryIds?: string[]`

12.7 UserSettings

Represents global extension behavior.

Fields:

- `isEnabled: boolean`
- `defaultApplyMode: ApplyMode`
- `promptToSaveNewAnswers: boolean`
- `autoApplyConfidenceThreshold: number`
- `showInlineIndicators: boolean`
- `siteRulesEnabled: boolean`

12.8 SiteRule

Represents host-specific behavior overrides.

Fields:

- `hostName: string`
- `isEnabled: boolean`
- `overrideApplyMode?: ApplyMode`

---

13. Enums

FieldType

- `Text`
- `TextArea`
- `Select`
- `Checkbox`
- `Radio`
- `Unknown`

AnswerType

- `Text`

- `Boolean`
- `SelectChoice`
- `MultiSelect`
- `Unknown`

ApplyMode

- `Disabled`
- `SuggestOnly`
- `AskBeforeApply`
- `AutoApply`

ConfidenceLevel

- `None`
- `Low`
- `Medium`
- `High`

SiteRuleMode

- `Default`
- `Disabled`
- `Override`

---

14. Service Responsibilities

QuestionNormalizationService

Normalizes raw question text into a stable comparable form.

Responsibilities:

- lowercase text
- trim whitespace
- remove or normalize punctuation
- collapse repeated spaces
- optionally extract useful keywords

MemoryMatchingService

Matches a `QuestionSignature` against all saved memories.

Responsibilities:

- exact normalized matching
- near-exact string similarity
- keyword overlap fallback
- field type compatibility checks

MemoryDecisionService

Determines what MemoryBank should do with a match.

Possible outcomes:

- do nothing
- suggest
- prompt
- auto-apply

MemorySaveService

Creates and stores new memories.

Responsibilities:

- normalize question text
- infer answer type
- detect duplicates
- persist memory metadata

SettingsResolutionService

Resolves effective behavior for the current site.

Responsibilities:

- combine global settings and site rule overrides
- determine effective apply mode
- determine whether MemoryBank is active on the page

MemorySearchService

Supports dashboard filtering and search.

DomScannerService

Scans the page for relevant form controls.

DomQuestionLocatorService

Extracts the likely question for a field using:

- label text
- aria attributes
- placeholder
- nearby sibling text
- section headings
- fieldset and legend text

DomFieldWriterService

Writes an answer into a field.

DomEventDispatcherService

Dispatches correct DOM events after writing values.

MutationObserverService

Watches for dynamic form changes and rerenders.

ChromeStorageService

Wraps local storage operations.

ChromeMessagingService

Wraps cross-runtime messaging.

LoggerService

Captures debug logging in development.

---

15. Repository Responsibilities

MemoryRepository

- get all memories
- get memory by id
- save memory
- update memory
- delete memory
- search memories

SettingsRepository

- load settings
- save settings

SiteRulesRepository

- load rules
- get rule by host
- save rule
- delete rule

---

16. View Responsibilities

Popup views

- `PopupHomeView`
- `PopupStatusCard`
- `PopupModeSelector`
- `PopupSiteToggleCard`

Responsibilities:

- enable or disable MemoryBank
- switch apply mode
- toggle current site behavior
- display active state

Dashboard views

- `MemoryDashboardView`
- `MemoryListView`
- `MemoryEditView`
- `SettingsView`

- `SiteRulesView`

Responsibilities:

- browse and search memories
- edit or delete memories
- manage global settings
- manage site-specific rules

In-page views

- `FieldSuggestionBadgeView`
- `ApplyPromptView`
- `SaveMemoryPromptView`
- `FieldStateIndicatorView`

Responsibilities:

- show a previous memory is available
- ask whether to apply a memory
- ask whether to save a new answer
- show field status subtly

---

17. Main Data Flows

Data Flow 1: Analyze and suggest

1. Content runtime scans the page.
2. A field is found.
3. Question text is extracted.
4. A `DetectedFieldDescriptor` is built.
5. Background runtime resolves effective settings.
6. Question normalization runs.
7. Memory matching runs.
8. Decision service determines the action.
9. Content runtime shows suggestion, prompt, or auto-apply state.

Data Flow 2: Apply a memory

1. User accepts a suggestion, or auto-apply is triggered.
2. Content runtime writes the value.

3. DOM events are dispatched.
4. Usage metadata is updated.

Data Flow 3: Save a new memory

1. User manually fills a field.
2. Content runtime detects meaningful completion.
3. Save prompt is shown.
4. User confirms.
5. Save service creates a new `MemoryEntry`.
6. Repository persists it locally.

Data Flow 4: Edit a memory

1. Dashboard loads memory list.
2. User selects a memory.
3. User edits question and answer.
4. Update flow normalizes the updated question.
5. Repository persists the new state.

---

18. Message Boundaries

MemoryBank should communicate through clear message contracts between runtimes.

Example message categories

- analyze page
- request match for field
- apply memory
- save memory
- load settings
- update settings
- fetch memories
- update memory
- delete memory
- fetch site rule
- update site rule

The project should avoid ad-hoc message payloads. Message shapes should be strongly typed from the beginning.

---

19. Matching Strategy

Final v1 decision

MemoryBank will use a **non-AI, local, explainable matching strategy**.

Matching stages

1. exact normalized match
2. near-exact string similarity
3. keyword overlap fallback
4. field type compatibility filter

Why this is the right choice

- keeps the product lightweight
- preserves privacy
- avoids model cost
- is easier to debug
- is easier to explain
- avoids AI fatigue entirely

---

20. Settings Spec

Global settings

- enabled
- default apply mode
- prompt to save new answers
- auto-apply confidence threshold
- show inline indicators
- enable site rules

Per-site settings

- site enabled or disabled
- site-specific apply mode override

Supported apply modes

- `SuggestOnly`

- `AskBeforeApply`
- `AutoApply`

Recommended default

- `AskBeforeApply`

This is the best balance between usefulness and safety.

---

21. Pain Points and Mitigations

Pain point 1: Question extraction is messy

Many sites do not properly associate labels with fields.

Mitigation:

- layered extraction strategy
- fallback heuristics
- keep extraction logic isolated in `DomQuestionLocatorService`
- add site-specific improvements later if needed

Pain point 2: Wrong autofills

A weak match can produce a bad suggestion or fill.

Mitigation:

- default to `AskBeforeApply`
- use confidence levels
- show source question before applying
- gate auto-apply by threshold

Pain point 3: Duplicate memories

Users may save similar memories many times.

Mitigation:

- duplicate checks during save
- editing support in dashboard
- merge support later if needed

Pain point 4: Noisy save prompts

Prompting too often can become annoying.

Mitigation:

- only prompt on meaningful completion
- minimum content thresholds
- prompt on blur or stable completion points
- ignore obviously trivial fields where appropriate

Pain point 5: Dynamic form rerenders

Modern sites often rerender fields.

Mitigation:

- mutation observer
- debounced re-analysis
- field identity strategy resilient to DOM changes

Pain point 6: Huge content script risk

It is easy for this kind of project to collapse into one giant content script.

Mitigation:

- keep content layer thin
- keep decisions in application layer
- keep storage behind repositories

---

22. Implementation Guardrails

These rules should remain true throughout development.

Rule 1

The content layer may detect and apply, but it should not own business logic.

Rule 2

The domain layer must not depend on Chrome APIs or raw DOM elements.

Rule 3

Storage access should happen through repositories only.

Rule 4

Settings resolution should happen in one place, not in multiple runtimes.

Rule 5

Matching behavior must remain explainable.

Rule 6

Any new feature must be evaluated against the lightweight and local-first philosophy.

---

23. MVP Build Order

Phase 1: Foundation

- manifest setup
- runtime messaging skeleton
- storage wrappers
- repositories
- popup enable or disable control
- popup mode selection

Phase 2: Detection

- DOM scanner
- question locator
- descriptor creation
- page analysis coordinator

Phase 3: Memory core

- normalization service
- matching service
- match result model
- settings resolution

Phase 4: Apply flow

- inline suggestion UI
- apply prompt
- field writing service
- DOM event dispatch support

Phase 5: Save flow

- manual completion detection
- save prompt
- save service
- duplicate handling basics

Phase 6: Dashboard

- memory listing
- search and filtering
- edit and delete memory
- settings page
- site rules page

Phase 7: Hardening

- dynamic form support improvements
- extraction heuristics refinement
- compatibility improvements on difficult sites

---

24. Final Product Positioning

MemoryBank should be positioned as:

> A lightweight, local-first form memory extension that remembers how you answered questions before and helps you reuse those answers quickly.

It should not be positioned as:

- an AI copilot
- a career platform
- a job search suite
- an autonomous browser agent

That focus is part of the product's strength.

---

25. Final Recommendation

The project should move into coding only after preserving these decisions:

- no AI anywhere
- local-only storage
- simple matching strategy
- thin content layer
- layered architecture
- editability and transparency over cleverness
- `AskBeforeApply` as default
- compact UI and limited scope

If MemoryBank stays inside those boundaries, it has a strong chance of being both useful and realistically shippable.